PROJECT-BASED LEARNING

PHASE-II EVALUATION

SmartFile : C++ Based File Compression and Decompression Using Huffman Coding

Team ID: DSCPP-III-2026-T056

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T056
Semester: 3rd
Section: DS1,ML1,ML-2,SEC-H
Domain: *AI-ML/AI-DS/ CSE CORE*

Team Members

1. *Rishi Negi – 2510380076*
2. *Vardan Bansal – 2510372517*
3. *Priyansh Rautela – 2510370501*
4. *Pragya Saksham – 2510370412*

Mentor

Dr.Hemant Singh Pokhariya

Interactions completed: 7

The team is developing SmartFile in C++ using Huffman Coding. It aims to compress files to save space and decompress them to recover the original data.

Phase-I Recap & Feedback Incorporated

Phase-I Status

During Phase-I, the team proposed SmartFile as a C++ application for lossless file compression and decompression using Huffman Coding.The basic system design, project requirements and main modules were identified.

Feedback Received

- *Improve the project architecture.*
- *Clearly define the main modules.*
- *Plan testing and validation.*

Changes Incorporated

- *Improved system design.*
- *Added file format and RAW mode.*
- *Planned file recovery testing..*

Phase-I planning was improved by defining clear modules, updating the system design, and planning testing and validation.

Problem Statement & Finalized Objectives

Problem Statement

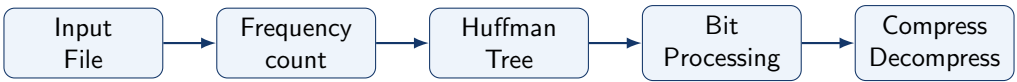
Large files can require more storage space. SmartFile aims to reduce file size using Huffman Coding while preserving the original data. The decompressed file should match the original file exactly.

Finalized Objectives

1. *Read files in binary mode and count the frequency of each byte.*
2. *Build a Huffman Tree using a min-heap priority queue.*
3. *Encode data into packed bits and save it in a custom .sfile format.*
4. *Decompress files and verify that the original data is recovered without changes*

Use RAW mode when compression increases file size due to metadata overhead.

Updated System Architecture / Workflow



Major Components

- *File handling and Frequency calculation*
- *Huffman Tree and code generation*
- *Bit processing and file output*

Changes Since Phase-I

- *More detailed module structure*
- *Planned custom file header and metadata*
- *Added RAW mode for cases where compression increases file size*

The modules process the input file step by step. During decompression, the saved frequency information is used to rebuild the Huffman Tree and recover the original bytes. The complete workflow is still under development.

Module Design & Functional Requirements

Major Modules

1. *File Handling*
2. *Frequency Calculation*
3. *Huffman Tree*
4. *Bit Processing*
5. *Compression & Decompression*

Functional Requirements

- *File Selection*
- *Byte Counting*
- *Code Generation*
- *Metadata Storage*
- *File Recovery*

Module Dependency

The input file is analyzed to build a frequency table and Huffman Tree. The generated codes are used to process bits and create the output file.

BitWriter has been implemented. BitReader and the decompression module are currently in progress.

Technology Stack & Development Methodology

Technology Stack

- Programming: *C++*
- Database: *Not Required*
- Framework: *Huffman Coding,File Handlling*
- Development Tool: *Visual Studio Code*
- Version Control: *Git / GitHub*

Development Methodology

1. Requirement Analysis
2. System Design
3. Huffman Implementation
4. Testing and Debugging
5. Performance Evaluation

C++ is used for file handling and DSA implementation. Huffman Coding helps compress files, while VS Code provides a development environment. Git/GitHub supports version control and team collaboration. The project is developed module by module, followed by testing and verification.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures	*Arrays, Linked Lists, Trees, Priority Queue*	*HuffMan Module*
OOPs with C++	*Classes,Object,Encapsulation,Function*	*Compression and Decompression*

SmartFile applies C++ and DSA concepts such as Huffman Coding, Priority Queue, and Binary Trees to implement file compression and decompression.

Development & Implementation Progress

Module	Status	Work Completed
Basic Structure and File Handling	Completed	*Initial menu and binary file handling*
Frequency Table and Huffman Tree	Completed	*Byte counting, min-heap and code generation*
Bit-Level Processing	In Progress	*BitWriter implemented; BitReader remains*
File Format and Decompression	Pending	*Planned work(Metadata, decompression and integration)*

Overall Project Completion: *45%*

The basic C++ structure, menu, frequency array, Huffman tree, code generation, and initial BitWriter have been implemented. Remaining work includes completing the .sfile format, BitReader, decompression, RAW mode, compression statistics, and testing with error handling.

Working Prototype / Demonstration

Implemented Features

- *Basic C++ project structure, menu, and file handling.*
- *Binary file reading/writing and byte-frequency calculation.*
- *Huffman Tree construction using a priority queue.*
- *Huffman code generation and initial BitWriter implementation with buffering and flushing*

Prototype / Output

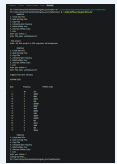


Figure: Huffman Coding Output

The current prototype demonstrates the initial implementation of the core data structures and file-processing components. Complete compression, decompression and round-trip verification are still pending.

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Compilation and File Handling	*Program runs and handles files correctly*	*Menu and small-file handling checked.*	PASS
Compression Statistics	*Correct file sizes and compression percentage.*	*Percentage calculation needs refinement.*	FAIL
Round-Trip Validation	*Decompressed file matches the original.*	*File comparison is pending.*	PENDING

Results / Validation

Initial testing confirms successful menu operation, frequency calculation, and Huffman code generation. Compression statistics require refinement, while complete decompression validation remain pending.

The results reflect the current development stage. Final performance and lossless recovery claims will be confirmed after integrated testing.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: *https://github.com/Vardanbansalgeu/pbl_sem3*
- README.md: *Created*
- Contributors: *All Members Added*
- Repository Status: *Created*

Development Activity

- *Code changes tracked using Git*
- *Commit messages maintained for updates.*
- *Team members work on assigned modules.*
- *Latest implementation uploaded to GitHub.*

The GitHub repository stores project files and tracks code changes. The repository, commit history, and contributors can be shown during the presentation.

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Rishi Negi	*Project-Planning*	*Requirements analysis and system design*	*1*
Vardan Bansal	*Developer*	*Huffman Tree, priority queue, and code generation*	*2*
Priyansh Rautela	*Developer*	*C++ menu, file handling, and BitWriter*	*1*
Pragya Saksham	*Testing*	*Initial testing and test-case preparation*	*1*

Each member's contribution is linked to specific project modules, such as Huffman Tree construction, file handling, BitWriter implementation, or test-case preparation. GitHub commit history should be used to verify individual contributions.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- *Data Processing*
- *Metadata Handling*
- *File Validation Testing*

Solutions / Learning

- *Binary file handling and RAW mode implementation*
- *Huffman Coding, bit-level processing, and file validation*

Phase-III Roadmap

1. *Complete metadata handling*
2. *Decompression module*
3. *Implement RAW mode and fix compression statistics*
4. *Test text, binary, small, and compressed files*
5. *Verify exact recovery and finalize documentation*

The main challenges are bit-level processing, metadata handling, and compression statistics. Phase-III will focus on completing decompression, implementing RAW mode, testing cases, and verifying exact file recovery.

Expected Outcomes & References

Expected Final Outcomes

- *Working C++ file compression and decompression system*
- *Compression ratio, file-size savings, and execution time*
- *Potential for handling different file types and larger files*
- *Scope for further research in compression techniques*

References

1. *C++ Standard Library Documentation*
2. *Huffman, D. A. — "A Method for the Construction of Minimum-Redundancy Codes"*
3. *C++ documentation for file streams, priority queues*

Thank You
Questions & Suggestions